

// GLOBEGRID — LEGENDARY TIER
// BUILDS ON: REV 1.2 MANUAL + v2 EXPANSION ADDENDUM (BOTH ASSUMED BUILT)

GLOBEGRID v3

LEGENDARY TIER — BUILD MANUAL

Self-arguing AI, forward-looking forecasts, provable trust, and unhinged graphics.

REVISION	3.3 — analyst panel added
ASSUMES	GlobeGrid v1 (zero-install SQLite/stdlib base) + v2 addendum, both fully built
EXCLUDED	The 'get it off the screen' category (OS tray, smart-home lighting, situation-room display) — removed per instruction, not in scope for v3
TARGET AGENT	Claude Code, phase-scoped, same CLAUDE.md continuity pattern as v1/v2
TONE NOTE	Section 6 (forecasting) carries real reputational risk if shipped carelessly — read its caveats before building, not after

This manual covers everything raised in the 'legendary tier' brainstorm — the wilder ideas past the v2 addendum's already-comprehensive scope — organized into four groups: AI that argues with itself, genuinely predictive (not just retrospective) capability, unhinged graphics, and provable trust. Every item is written at the same level of technical detail as the v1 manual and v2 addendum: problem statement, mechanism, schema, and config where applicable.

SECTION 01 // SCOPE & ASSUMPTIONS

Scope and assumptions

This document assumes the v1 base (SQLite/stdlib zero-install backend, buildless JS frontend, real sentence-transformers embeddings active) and the full v2 expansion addendum (NER + entity canonicalization, near-duplicate detection, prediction scorecard, time capsule mechanism, expanded data sources, graphics v2, analytics tools, resilience/ops upgrades, and the offline gazetteer) are already built and working. Nothing in this document restates v1/v2 mechanics except where a v3 feature directly extends one.

1.1 What's excluded

The 'get it off the screen entirely' category from the source brainstorm — OS tray mode, Philips Hue smart-lighting integration, and the Raspberry Pi 'situation room' companion display — is removed per instruction. None of those three appear anywhere below.

1.2 What's in scope

- **Group A — AI that argues with itself:** multi-agent debate, devil's-advocate pass, cross-lingual embedding space, self-tuning correlation thresholds, anomaly/changepoint detection on the instability index.
- **Group B — Genuinely predictive:** forward-looking risk forecasting, the butterfly-effect lineage view.
- **Group C — Unhinged graphics:** WebXR mode, generative reactive music, auto-rendered timelapse export, real satellite overlay.
- **Group D — Provable trust:** hash-chained fact provenance, story version history.
- **Group E — Geopolitical entity layer:** country profiles, alliances and blocs, ongoing conflicts, non-state actors, marked cities/locations and critical infrastructure, international organizations, bilateral relations, sanctions regimes, treaties and agreements, notable persons, and an elections tracker.
- **Group F — Analyst panel:** an LLM-powered natural-language Q&A surface grounded in GlobeGrid's own data, with full citation back to source stories.

As with the v2 addendum: every item below is in scope. Section 15's build sequence is for dependency ordering only, following the same three-picks-first guidance given when this brainstorm was proposed (butterfly-effect lineage, devil's advocate, hash-chained provenance) with forecasting deliberately last.

SECTION 02 // MULTI-AGENT DEBATE

Multi-agent debate causal engine

Problem: a single causal-link call (v1 Section 9) produces one narrative and one confidence label. Disagreement between plausible readings of the same evidence is itself informative and is currently thrown away — the system only ever shows its one conclusion, never its uncertainty about competing explanations.

2.1 Mechanism

- Run the Section 9 cluster payload through three independent Claude calls, each with a distinct persona system-prompt prefix appended before the existing Section 9 rules: **Skeptic** (actively looks for reasons the causal claim might be wrong or overstated), **Historian** (weighs the cluster against similar historical patterns in the fact chain), **Optimist** (the most charitable causal reading the evidence actually supports — not blind positivity, just the least-skeptical honest reading).
- All three calls receive the identical cluster payload — the variable is the persona framing, not the input data.
- Store all three outputs. The **primary** displayed narrative remains the existing single-call output (unchanged behavior for anyone not looking closer) — the debate is an additional layer, not a replacement.
- Compute a **disagreement_score**: pairwise embedding distance between the three personas' **cause** strings, averaged. High disagreement is surfaced in the UI as a distinct visual state — not hidden inside a number nobody reads.

New table: **causal_debate**

Field	Type	Description
id	uuid PK	Primary key
story_id	uuid FK	References stories.id
persona	text	skeptic historian optimist
narrative	text (JSON)	Same shape as stories.causal_narrative (Section 9)
generated_at	timestampz	When this persona's call ran

- Add **stories.disagreement_score** (REAL, 0-1) and **stories.debate_generated_at**.
- Runs on the same trigger as the primary causal link (new/updated cluster), but as a separate, independently-failing step — a debate-generation failure never blocks the primary narrative from displaying (same failure-isolation discipline as v1 Section 10.2).

SECTION 03 // DEVIL'S ADVOCATE

Built-in devil's-advocate pass

Problem: a single-pass 'high confidence' label can be false certainty — the prompt has no structural requirement to actually stress-test its own conclusion before stating it.

3.1 Mechanism

- After the primary Section 9 narrative is generated (and independent of the multi-agent debate in Section 2, which can run alongside or be skipped — this pass is cheaper and more targeted), issue one additional Claude call: given the primary narrative and the same cluster facts, answer specifically '*what would have to be false for this causal claim to be wrong?*'
- The response is structured JSON: `{counter_argument: string, weakens_confidence: boolean, revised_confidence: "high"|"medium"|"low"|null}`.
- If `weakens_confidence` is true, the stored `stories.confidence` is downgraded to `revised_confidence` — the devil's-advocate pass can only ever lower confidence, never raise it, since its entire job is to catch overclaiming, not to add false reassurance.
- The counter-argument itself is stored and surfaced on the story page directly beneath the primary narrative, labeled plainly (e.g. 'A counter-reading:') — not hidden as backend-only metadata.
- Add `stories.counter_argument` (text, nullable) and `stories.confidence_pre_devil_advocate` (text, nullable) so the downgrade is auditable — you can always see what the confidence was before this pass touched it.

This is one of the three highest-value, lowest-cost items in this entire document per the original brainstorm framing — small build, direct improvement to trustworthiness.

SECTION 04 // CROSS-LINGUAL EMBEDDING

Cross-lingual embedding space

Problem: correlation currently depends on English-normalized descriptions. A Xinhua article and a BBC article about the same event only correlate today if one has been translated first — and translation (v1 Section 5.8) is explicitly display-only, not part of the correlation pipeline.

4.1 Mechanism

- Swap the embedding model from `all-MiniLM-L6-v2` to a multilingual sentence embedding model (e.g. `paraphrase-multilingual-MiniLM-L12-v2`) — same sentence-transformers library, same auto-detect/fallback pattern already established in v1's `embed.py`, just a different model name in config.
- This lets non-English facts (from the v2 addendum's Xinhua/TASS/Le Monde/Haaretz/Times of India sources) correlate *directly* against English facts by meaning, without translation as a prerequisite.
- Translation (v1 Section 5.8) is unchanged and stays display-only — this is purely a correlation-quality upgrade, not a UI change.
- Triggers the existing `ensure_embedder_consistency()` re-embed-the-whole-chain routine from v1 — this swap is exactly the kind of embedder change that mechanism was built to handle gracefully.

Config change only: `EMBEDDING_MODEL_NAME=paraphrase-multilingual-MiniLM-L12-v2` in `.env`. No schema change — the embedding column is already a fixed-width vector; confirm the multilingual model's output dimension matches (384 for the model named above) or adjust `EMBEDDING_DIM` accordingly if a different multilingual model is substituted.

SECTION 05 // SELF-TUNING THRESHOLDS

Self-tuning correlation thresholds

Problem: the correlation thresholds (v1 Section 7.2 — 0.78 same-window, 0.85 historical) are static config values, tuned once by guesswork and never revisited. The system has no mechanism to learn from its own mistakes.

5.1 Mechanism

- Add a thumbs-up/down control on each story page: 'was this really the same story?' — a lightweight, one-tap feedback signal, not a survey.
- Each vote is stored per story, tagged with the event **category** (v1 Section 6.3) at the time of voting, since different categories likely warrant different thresholds (e.g. finance stories may cluster more precisely than geopolitics ones).
- A scheduled job aggregates recent feedback per category and nudges that category's effective threshold by a small, bounded step (e.g. ± 0.01 per aggregation cycle, clamped to a configured min/max range) — a simple exponential-moving-average adjustment, not a full learned model. Deliberately conservative: this should drift slowly, not swing on a handful of votes.
- Effective per-category thresholds override the global default from config, but the global default remains the floor/ceiling — self-tuning narrows within a safe band, it doesn't escape it.

New table: `correlation_feedback`

Field	Type	Description
<code>id</code>	<code>uuid PK</code>	Primary key
<code>story_id</code>	<code>uuid FK</code>	References <code>stories.id</code>
<code>category</code>	<code>text</code>	Event category at time of vote (v1 Section 6.3 categories)
<code>vote</code>	<code>text</code>	correct incorrect
<code>voted_at</code>	<code>timestampz</code>	Timestamp

New table: `category_thresholds`

Field	Type	Description
<code>category</code>	<code>text PK</code>	Matches <code>events.category</code>
<code>same_window_threshold</code>	<code>REAL</code>	Current effective value, initialized from config default
<code>historical_threshold</code>	<code>REAL</code>	Current effective value, initialized from config default
<code>last_adjusted_at</code>	<code>timestampz</code>	When the tuning job last moved this value

SECTION 06 // ANOMALY DETECTION

Anomaly / changepoint detection on the instability index

Problem: the instability index (v1 Section 5.6) is a number on a trend line — 'is this unusual?' currently requires a human to eyeball the sparkline.

6.1 Mechanism

- Stdlib-compatible approach (consistent with the zero-install build): a rolling z-score over the instability history — compare the latest reading against the mean/standard deviation of a trailing window (config-defined, default 30 days of readings), flag when the z-score exceeds a threshold (default 2.5).
- For genuine changepoint detection (a sustained shift, not just one spike) rather than simple outlier flagging: a basic CUSUM (cumulative sum control chart) pass over the same rolling window — small enough to hand-roll in pure Python, no new dependency required.
- Anomalies are flagged, not acted on automatically — this feeds a visible marker on the instability trend line and an entry in a lightweight anomaly log; it does not trigger any automated notification (that category of feature is explicitly out of scope per Section 1.2).

New table: `anomaly_flags`

Field	Type	Description
<code>id</code>	<code>uuid PK</code>	Primary key
<code>detected_at</code>	<code>timestampz</code>	When the anomaly was flagged
<code>method</code>	<code>text</code>	zscore cusum
<code>score_value</code>	<code>REAL</code>	The <code>instability_scores</code> value that triggered the flag
<code>z_or_cusum_value</code>	<code>REAL</code>	The statistic that crossed threshold

SECTION 07 // RISK FORECASTING

Forward-looking risk forecasting

This is explicitly flagged, in the source brainstorm and here, as the highest reputational risk item in this document. Overclaiming predictive power is a real hazard for a tool like this. Build this last, and read this entire section before writing a line of it.

7.1 What this is

Extend the causal engine to occasionally state a short-horizon risk statement — e.g. 'elevated risk of X in region Y over the next 72 hours' — heavily caveated, and critically, **tied directly into the v2 addendum's prediction scorecard (v2 Section 3.4)** so every forecast is graded later against what actually happened. This is what makes the feature defensible rather than reckless: it is provably honest over time, not just confident-sounding in the moment.

7.2 Mechanism

- Forecasts are generated by a distinct, separately-labeled prompt path — never silently blended into the existing Section 9 causal narrative. A forecast is a clearly different kind of claim (about the future) than a causal narrative (about the past/present), and the UI must never let them look like the same thing.
- Every generated forecast is inserted into the existing `predictions` table (v2 Section 3.4) with a new `kind` column distinguishing `retrospective_consequence` (the existing v2 behavior) from `forward_forecast` (new).
- Forecast confidence is capped lower than causal-narrative confidence by policy — this system has no track record yet, so nothing should ever be labeled 'high confidence' for a forward forecast until the scorecard has enough resolved history to justify it (a config-defined minimum resolved-forecast count before 'high' becomes reachable at all).
- The UI must display every forecast alongside its own historical accuracy rate once available ('this system's 72h forecasts have been directionally correct N% of the time, based on M resolved forecasts') — a forecast without its own track record attached is not shippable under this spec.

7.3 Schema (extends v2's predictions table)

- Add `predictions.kind` (text: `retrospective_consequence` | `forward_forecast`).
- Add `predictions.horizon_hours` (integer, nullable — only set for `forward_forecast` rows).
- Add `predictions.region` (text, nullable) for forecast-specific scoping.

Sequencing requirement, not a suggestion: do not build this before the prediction scorecard (v2 Section 3.4) has real, resolved history to point to. A forecasting feature with zero track record is the single most reputationally exposed thing in either document.

SECTION 08 // LINEAGE VIEW

Butterfly-effect lineage view

Distinct from the v2 addendum's node-link graph explorer (v2 Section 6.3), which shows *co-occurrence* — entities that appeared together. This is *single-thread ancestry*: pick one fact, and render everything that fact ever fed into, growing forward through time. The most visually poetic use of 'facts are never deleted' anywhere in the system, and one of the cheapest builds in this document.

8.1 Mechanism

- The lineage is already implicit in existing data — `story_members` links facts/events to stories, and the historical-chain correlation path (v1 Section 5.4) already records exactly which past fact caused a new event to join a cluster.
- Add one new table that makes this explicit and queryable in one direction, rather than reconstructing it from joins every time: `lineage_edges`, populated at the same moment the correlation engine (v1 `correlate.py`) creates a `historical_chain` story_member — that is the exact event that represents one fact influencing a future one.
- Frontend: given a starting `fact_id`, walk `lineage_edges` forward (breadth-first) and render as a growing tree/timeline — a simple recursive layout is sufficient, no new rendering framework needed, reusing the same hand-rolled canvas approach as the v2 graph explorer.

New table: `lineage_edges`

Field	Type	Description
<code>id</code>	<code>uuid PK</code>	Primary key
<code>from_fact_id</code>	<code>uuid FK</code>	The earlier fact
<code>to_fact_id</code>	<code>uuid FK</code>	The later fact it influenced, via a shared story
<code>via_story_id</code>	<code>uuid FK</code>	The story cluster that connected them
<code>created_at</code>	<code>timestampz</code>	When this edge was recorded (matches the <code>historical_chain</code> link time)

SECTION 09 // WEBXR MODE

WebXR room-scale mode

Stand inside the globe in room-scale VR and walk through instability hotspots. Genuinely buildable directly on top of the existing Tier 1 WebGL2 renderer via the WebXR Device API — no new rendering engine, no new dependency.

9.1 Mechanism

- Request an `immersive-vr` XR session from the existing Tier 1 renderer's WebGL2 context; WebXR provides the head-tracked view/projection matrices that replace the existing orbit-camera controls while a session is active.
- Scale consideration: the globe as currently rendered is a small object viewed from outside. Room-scale WebXR needs a scale/mode toggle — either shrink the user (walk on a tabletop-sized globe) or invert the model so the user stands inside a sphere with the map projected on the interior surface (the more literal reading of 'walk through instability hotspots'). Both are valid; pick the interior-surface mode as the default since it matches the brainstorm's framing most directly.
- Existing effects (Section 5 of the v2 addendum — bloom, particle trails, day/night terminator) all continue to render normally in an XR session; no separate XR-specific asset pipeline is needed.
- Gracefully absent on non-XR-capable devices — feature-detect `navigator.xr` and simply omit the 'Enter VR' control when unsupported, consistent with the existing tiered-degradation philosophy from v1 Section 11.

9.2 Generative reactive music

Replaces the v2 addendum's static ambient-loop-plus-blip (v2 Section 5.2) with actual procedural composition.

- Web Audio API oscillators/sequencer, driven by two live inputs: the current instability score (maps to key/tempo — higher instability, more dissonant/faster) and per-region event density (maps to which regions get an active 'instrument voice' at any given moment).
- A simple generative approach is sufficient and stays dependency-free: a small set of pre-defined scales/chord pools selected by instability band, a step-sequencer clocked to a tempo derived from the same score, and regional voices panned to roughly match their position on the globe (stereo/spatial panning tied to the camera's current view direction).
- The v2 mute-by-default and single persistent toggle behavior carries over unchanged.

SECTION 10 // TIMELAPSE & SATELLITES

Timelapse export and satellite overlay

10.1 Auto-rendered timelapse export

- Built directly on top of two things that already exist: the time-capsule scrubber (v2 Section 4) for stepping through history, and the idle 'tour' camera (v2 Section 5.1) for automatic movement.
- Client-side render: drive the scrubber programmatically across a chosen time range, capture each frame via [WebCodecs](#) (VideoEncoder), and mux to MP4 entirely in-browser — no server-side rendering pipeline needed.
- Output: a downloadable 'last 24 hours of the world' (or any as_of range) MP4 — the only feature in either document that produces a shareable artifact outside the app itself.
- Practical constraint: rendering at real interactive frame rate isn't necessary for export — the capture loop can run slower than realtime (render frame, encode, advance scrubber, repeat) since it's not being watched live during capture, which keeps this feasible even at Tier 1 visual quality.

10.2 Real satellite overlay

- Public TLE (Two-Line Element) orbital data from CelesTrak — free, no key, updated regularly.
- Client-side SGP4 propagation (a compact, well-documented algorithm; a small pure-JS implementation is sufficient, no server-side orbital computation needed) converts TLE data into live lat/lon/altitude for each tracked satellite.
- Rendered as small moving points at their actual altitude above the globe surface (distinct visually from ground-level event pins) — purely an aesthetic/informational layer, not correlated with any other data in v3.
- Fetch TLE data on a slow schedule (daily is plenty — orbital elements don't change fast enough to need more) and propagate positions client-side in real time between fetches.

SECTION 11 // HASH-CHAINED PROVENANCE

Hash-chained fact provenance

This is what makes the prediction scorecard (v2 Section 3.4) and the forecasting feature (Section 7 above) actually credible instead of self-reported — proof a prediction was logged before the event happened, not quietly edited after the fact.

11.1 Mechanism

- Append-only hash chain over `extracted_facts` (and, separately, over `predictions`, since that's the table whose integrity actually matters most): each row's hash is computed over its own content plus the previous row's hash, so altering any historical row breaks every hash after it.
- This requires no new infrastructure — a single SHA-256 computation at insert time, stdlib `hashlib`, no blockchain, no external service.
- A verification script (`scripts/verify_provenance.py`) walks the chain and confirms every hash matches — run on demand, and ideally surfaced as a small 'chain verified ✓' indicator somewhere in the UI so the guarantee is visible, not just theoretically true.

Schema additions

Field	Type	Description
<code>extracted_facts.row_hash</code>	<code>text</code>	SHA-256 of this row's content + previous row's hash
<code>extracted_facts.prev_hash</code>	<code>text</code>	Hash of the immediately preceding row in insertion order
<code>predictions.row_hash</code>	<code>text</code>	Same pattern, applied to the predictions table specifically
<code>predictions.prev_hash</code>	<code>text</code>	Same pattern

Important scope note: this proves the chain wasn't altered after being written by this system — it does not, by itself, prove the underlying source data was true. That distinction should be stated plainly anywhere this guarantee is surfaced to avoid overclaiming what it actually verifies.

SECTION 12 // STORY VERSION HISTORY

Story version history

Problem: `stories.causal_narrative` is overwritten on every regeneration (v1 Section 5.5) — the system's earlier read on a story, and how it changed as more facts came in, is currently lost.

12.1 Mechanism

- Before any update to `stories.causal_narrative`, insert the outgoing version into a new history table rather than discarding it.
- Frontend: a git-like diff view on the story page — select any two versions, see cause/affected/consequences rendered side by side with changed spans highlighted. A simple word-level diff (e.g. a Python `difflib`-based comparison, `stdlib`, no new dependency) is sufficient; this doesn't need a full structural diff algorithm.

New table: `story_narrative_versions`

Field	Type	Description
<code>id</code>	<code>uuid PK</code>	Primary key
<code>story_id</code>	<code>uuid FK</code>	References <code>stories.id</code>
<code>causal_narrative</code>	<code>text (JSON)</code>	The narrative as it stood before this version was superseded
<code>confidence</code>	<code>text</code>	Confidence level at that version
<code>superseded_at</code>	<code>timestampz</code>	When this version was replaced

SECTION 13 // COUNTRY PROFILES

Country profiles

A persistent, queryable profile per country — clickable directly from the map and globe, and reachable via a separate searchable directory. Real-time-updated where the underlying data actually changes in real time; slow-refreshed where it doesn't. The two should not be conflated.

13.1 What a profile contains

- **Leadership:** current head of state / head of government, ruling party.
- **Legislature:** chamber(s), ruling coalition, a plain-language composition summary.
- **Geopolitical and economic agendas:** a synthesized 'what this country appears to be pursuing right now' summary.
- **Market status and trade:** relevant index/currency snapshot, major trade partners, key exports.
- **Stances:** what the country's public positions/goals appear to be on active situations, drawn from its own tagged coverage.

13.2 Mechanism — two different update models, not one

- **Slow-changing reference facts** (capital, government type, current leader, ruling party) are seeded and periodically refreshed (default weekly, config-driven) from Wikidata's free structured API — these genuinely don't change often, and treating them as real-time would just mean re-fetching the same answer.
- **Agendas, economic agendas, and stance summaries are not scraped from any single source.** They are AI-synthesized, reusing the exact pattern already established for the v2 daily briefing (v2 Section 6.1): periodically aggregate that country's recently tagged facts/stories (via the canonical-entity link, v2 Section 3.1) and generate a fresh synthesis. This is the correct architecture here — the fact chain and causal narratives already contain the raw material; a country profile is a *view* over existing data, not a new ingestion source.
- **Market/trade data** reuses the existing Alpha Vantage integration (v2 Section 1.4) scoped to that country's major index/currency, plus a slow-refreshed trade-statistics snapshot from the World Bank Open Data API (free, no key).
- Every synthesized field stores which source story/fact IDs it was generated from, so — consistent with the attribution requirement running through this entire project — a profile's agenda summary is never just an assertion with nothing behind it.

13.3 Map/globe click interaction

- Load a simplified country boundary dataset client-side (Natural Earth 1:110m scale, public domain, small file size — appropriate for a globe/map at this zoom level, no need for high-resolution borders).
- On click: a straightforward point-in-polygon test against the loaded boundaries identifies which country was clicked, opening that country's profile panel — this needs no server round-trip beyond fetching the profile data itself.

- A separate, fully searchable country directory (extends the v2 full-text search, Section 6.4) lists every country by name/leader/region for anyone who'd rather search than click.

New table: countries

Field	Type	Description
id	text PK	ISO 3166-1 alpha-3 code
name	text	Common name
capital	text	
region	text	Geographic region grouping
government_type	text	
boundary_ref	text	Key into the loaded Natural Earth boundary dataset

New table: country_leadership

Field	Type	Description
country_id	text FK	References countries.id
role	text	head_of_state head_of_government
name	text	
party	text	Nullable
since_date	date	
last_refreshed_at	timestampz	Last Wikidata sync

New table: country_legislature

Field	Type	Description
country_id	text FK	References countries.id
chamber_name	text	
ruling_coalition	text	Plain-language summary
composition_summary	text	
last_refreshed_at	timestampz	

New table: country_agenda_synthesis

Field	Type	Description
country_id	text FK	References countries.id
geopolitical_agenda	text	AI-synthesized
economic_agenda	text	AI-synthesized
stance_summary	text	AI-synthesized
source_story_ids	text (JSON array)	Which stories this synthesis drew from — attribution, not just assertion
generated_at	timestampz	

New table: country_trade_stats

Field	Type	Description
country_id	text FK	References countries.id
gdp_usd	numeric	Nullable
major_trade_partners	text (JSON array)	
key_exports	text (JSON array)	
as_of_date	date	

SECTION 14 // ALLIANCES & BLOCS

Alliances and blocs

NATO, the EU, ASEAN, CSTO, BRICS, OPEC, the AU, and similar — represented as first-class entities in their own right, not just implied by co-membership.

14.1 Mechanism

- Seeded from Wikidata, same source and refresh cadence as country reference data (Section 13.2) — alliance membership changes rarely enough that a weekly sync is more than sufficient.
- Each country profile (Section 13) gains a visible 'Member of: ...' list.
- The map/globe gains a togglable overlay that tints country boundaries by shared bloc membership — off by default, since it's a distinct analytical view rather than the default map state.

New table: alliances

Field	Type	Description
id	uuid PK	
name	text	e.g. 'NATO'
type	text	military economic political regional
founded_date	date	Nullable
description	text	

New table: alliance_memberships

Field	Type	Description
alliance_id	uuid FK	References alliances.id
country_id	text FK	References countries.id
status	text	member observer candidate
joined_date	date	Nullable

SECTION 15 // ONGOING CONFLICTS

Ongoing conflicts

A dedicated tabbed view — each active conflict gets its own filtered slice of the existing live feed, rather than conflicts being an implicit pattern someone has to notice in the general feed.

15.1 Mechanism

- New nullable `conflict_id` column on both `events` and `stories` (extends the v1 schema).
- Auto-suggestion, not full automation: when a new event/story shares canonical entities (v2 Section 3.1) with a conflict's registered parties (below), suggest that conflict tag with a confidence flag rather than silently auto-assigning it — misclassifying a story into the wrong conflict is worse than a brief unclassified state. A lightweight confirm step (one click) resolves suggested tags.
- Each conflict tab is, mechanically, just the existing live feed (v1 Section 5.1) filtered by `conflict_id` — no new feed infrastructure required.
- Conflict parties explicitly include both countries and non-state actors (Section 16) via a single join table, since real conflicts routinely involve both.

New table: `conflicts`

Field	Type	Description
<code>id</code>	<code>uuid</code> PK	
<code>name</code>	<code>text</code>	
<code>region</code>	<code>text</code>	
<code>started_at</code>	<code>date</code>	
<code>status</code>	<code>text</code>	active ceasefire frozen resolved
<code>summary</code>	<code>text</code>	AI-synthesized, same pattern as Section 13.2's agenda synthesis
<code>last_updated_at</code>	<code>timestamp</code> tz	

New table: `conflict_parties`

Field	Type	Description
<code>conflict_id</code>	<code>uuid</code> FK	References <code>conflicts.id</code>
<code>party_type</code>	<code>text</code>	country non_state_actor
<code>country_id</code>	<code>text</code> FK	Nullable — set when <code>party_type</code> = country
<code>non_state_actor_id</code>	<code>uuid</code> FK	Nullable — set when <code>party_type</code> = non_state_actor (Section 16)
<code>role</code>	<code>text</code>	e.g. belligerent mediator backer

SECTION 16 // NON-STATE ACTORS

Non-state actors

Militant, insurgent, and similar non-state groups (the brainstorm's examples: al-Qaeda in Mali, Hezbollah) represented with the same rigor as countries — this is standard practice in open-source geopolitical tracking (the same category of entity Wikipedia, GDELT's own actor coding, and conflict trackers like ACLED and Liveuamap already represent publicly). Scope stays descriptive and attributive, consistent with everything else in this system: names, type, region, publicly reported affiliations, and a synthesis of what's already in the public record via ingested coverage — never operational or targeting detail of any kind.

16.1 Mechanism

- Each non-state actor is registered as its own canonical entity (v2 Section 3.1), so ordinary extraction and correlation naturally associate news mentions with it — no separate NLP pipeline needed.
- Profile page mirrors the country profile (Section 13) in spirit but with different fields: no government/legislature, instead a nullable affiliated-state link (e.g. a state-sponsorship relationship reported in the public record) and the conflicts it's a registered party to (Section 15).
- The `description_synthesis` field follows the exact same attribution discipline as Section 13.2 — generated from and linked back to specific ingested stories, never an unsourced assertion.

New table: `non_state_actors`

Field	Type	Description
<code>id</code>	<code>uuid PK</code>	
<code>name</code>	<code>text</code>	
<code>actor_type</code>	<code>text</code>	<code>militant terrorist insurgent cartel political_movement other</code>
<code>primary_region</code>	<code>text</code>	
<code>affiliated_state_id</code>	<code>text FK</code>	Nullable — references <code>countries.id</code>
<code>active_since</code>	<code>date</code>	Nullable
<code>canonical_entity_id</code>	<code>uuid FK</code>	Links into v2's <code>canonical_entities</code> (Section 3.1)
<code>description_synthesis</code>	<code>text</code>	AI-synthesized, source-linked per Section 13.2's pattern
<code>last_updated_at</code>	<code>timestampz</code>	

SECTION 17 // MARKED LOCATIONS

Marked cities and locations

Distinct from two things that already exist and are easy to confuse this with: the gazetteer (v2 Section 10, which resolves free-text place names to coordinates for extraction) and live event pins (transient, one per ingested event). This is a curated, persistent layer — capitals, active conflict zones, strategic chokepoints (e.g. the Strait of Hormuz, the Suez Canal, the Taiwan Strait), and contested territories — that stays on the map regardless of what's currently being ingested.

17.1 Mechanism

- Rendered as its own togglable layer, visually distinct from live event pins (a fixed marker style, not the pulsing/live-feeling treatment events get) so the two are never confused at a glance.
- Capitals populate automatically from `countries` (Section 13); conflict zones link to `conflicts` (Section 15); chokepoints and contested territories are a small manually-curated seed list, since there's no clean free API for 'globally significant strategic locations' the way there is for countries or alliances.

New table: `marked_locations`

Field	Type	Description
<code>id</code>	<code>uuid PK</code>	
<code>name</code>	<code>text</code>	
<code>lat</code>	<code>REAL</code>	
<code>lon</code>	<code>REAL</code>	
<code>category</code>	<code>text</code>	<code>capital</code> <code>conflict_zone</code> <code>strategic_chokepoint</code> <code>contested_territory</code> <code>other</code>
<code>country_id</code>	<code>text FK</code>	Nullable
<code>conflict_id</code>	<code>uuid FK</code>	Nullable
<code>description</code>	<code>text</code>	

17.2 Critical infrastructure and resources (extends the same table)

The 'more like this' category of strategic significance isn't only geographic — semiconductor fabs, rare earth mineral sites, undersea internet cables, energy pipelines, and LNG terminals matter for the same reason chokepoints do. Rather than a parallel table, these are additional `marked_locations.category` values, since the mechanism (a persistent, curated, point-on-map layer) is identical.

- Extended category set: adds `semiconductor_fab` | `rare_earth_site` | `undersea_cable` | `energy_pipeline` | `lng_terminal` to the enum already defined in 17.1.
- Same manually-curated seeding approach as chokepoints — no clean single free API covers this category comprehensively, so it starts as a small hand-built seed list (e.g. TSMC's Hsinchu/Tainan fabs, major rare-earth sites in China/Australia, primary transatlantic cable landing points) and grows over time.

SECTION 18 // INTERNATIONAL ORGANIZATIONS

International organizations

The UN, WHO, WTO, IMF, World Bank, ICC, and ICJ — distinct from alliances (Section 14), which are membership-based blocs with shared interests. These are global institutions with a mandate, and their statements/rulings are a primary-source signal in their own right, extending the same logic behind the v2 addendum's government-press-release sources (v2 Section 1.2) up to the multilateral level.

18.1 Mechanism

- Each organization is registered as its own canonical entity (v2 Section 3.1), exactly like non-state actors (Section 16) — this means ordinary extraction and correlation naturally associate coverage with it, with no separate ingestion path required.
- Where a clean official feed exists (UN press releases, WHO disease outbreak news, IMF country reports), register it as a proper source in the existing `sources` table (v1 Section 6.1) rather than relying on secondhand coverage alone.
- A profile page mirrors the country profile pattern: mandate summary, headquarters, and a synthesized 'recent posture' summary generated the same way as country agendas (Section 13.2) — source-linked, never an unsourced assertion.

New table: `international_organizations`

Field	Type	Description
<code>id</code>	<code>uuid PK</code>	
<code>name</code>	<code>text</code>	e.g. 'World Health Organization'
<code>org_type</code>	<code>text</code>	intergovernmental financial health judicial other
<code>mandate_summary</code>	<code>text</code>	
<code>hq_location</code>	<code>text</code>	
<code>canonical_entity_id</code>	<code>uuid FK</code>	Links into v2's <code>canonical_entities</code> (Section 3.1)
<code>founded_date</code>	<code>date</code>	Nullable

SECTION 19 // BILATERAL RELATIONS

Bilateral relations tracker

Country profiles (Section 13) describe a country in isolation. This tracks the relationship *between* any two countries — is the US-China relationship currently cooperative, tense, or openly hostile — which is often more informative than either country's profile alone.

19.1 Mechanism

- A scheduled job scans recent stories for country pairs that are both canonical-entity-tagged on the same story (v2 Section 3.1) and synthesizes/updates a relationship status and short narrative for that pair — same source-linked synthesis discipline as everything else in this layer.
- Displayed two ways: a small panel on each country's own profile ('Relations: tense with X, allied with Y'), and a dedicated relations-matrix view for anyone who wants the full picture at once — a simple grid/heatmap, not a new graph engine.

New table: `bilateral_relations`

Field	Type	Description
<code>id</code>	<code>uuid PK</code>	
<code>country_a_id</code>	<code>text FK</code>	References <code>countries.id</code>
<code>country_b_id</code>	<code>text FK</code>	References <code>countries.id</code>
<code>status</code>	<code>text</code>	<code>allied cooperative neutral tense hostile conflict</code>
<code>synthesis</code>	<code>text</code>	AI-synthesized narrative
<code>source_story_ids</code>	<code>text (JSON array)</code>	Attribution, same pattern as Section 13.2
<code>last_updated_at</code>	<code>timestampz</code>	

SECTION 20 // SANCTIONS REGIMES

Sanctions regimes

Who is sanctioning whom, for what, and since when — a distinct, structured signal that feeds directly into a country’s economic-agenda synthesis (Section 13.2) rather than staying buried in prose coverage.

New table: sanctions

Field	Type	Description
id	uuid PK	
imposing_party_type	text	country alliance international_organization
imposing_party_id	text	FK depends on imposing_party_type
target_country_id	text FK	Nullable
target_non_state_actor_id	uuid FK	Nullable
reason	text	
imposed_at	date	
status	text	active lifted
source_story_ids	text (JSON array)	

SECTION 21 // TREATIES & AGREEMENTS

Treaties and agreements

Trade deals, nuclear non-proliferation agreements, peace accords, environmental treaties — with actual signatory and ratification status per country, not just a name and a date.

New table: treaties

Field	Type	Description
id	uuid PK	
name	text	
treaty_type	text	trade nuclear_nonproliferation peace_accord environmental other
signed_at	date	
status	text	in_force signed_not_ratified withdrawn expired
summary	text	

New table: treaty_signatories

Field	Type	Description
treaty_id	uuid FK	References treaties.id
country_id	text FK	References countries.id
ratified	boolean	
ratified_date	date	Nullable

SECTION 22 // NOTABLE PERSONS

Notable persons

Country profiles (Section 13) cover the head of state/government. Real geopolitical signal often comes from figures below that level — foreign ministers, central bank governors, military commanders, opposition leaders — who currently have no representation anywhere in this system.

22.1 Mechanism

- Same canonical-entity linkage as non-state actors and international organizations — register the person once, extraction and correlation handle the rest.
- A person can affiliate with a country, an international organization, or a non-state actor (exactly one, via three nullable FKs) — covering officials, institutional figures, and non-state leadership alike with one table.

New table: notable_persons

Field	Type	Description
id	uuid PK	
name	text	
role_title	text	e.g. 'Foreign Minister', 'Central Bank Governor'
affiliated_country_id	text FK	Nullable
affiliated_org_id	uuid FK	Nullable — references international_organizations.id
affiliated_non_state_actor_id	uuid FK	Nullable
canonical_entity_id	uuid FK	Links into v2's canonical_entities
bio_summary	text	AI-synthesized, source-linked

SECTION 23 // ELECTIONS TRACKER

Elections tracker

Upcoming and recent elections per country — and, usefully, a trigger point: a completed election is exactly when a country's leadership data (Section 13) is most likely stale and most worth refreshing immediately rather than waiting for the next scheduled weekly sync.

23.1 Mechanism

- On an election's status changing to **completed**, immediately trigger a one-off **country_leadership** refresh for that country (Section 13.2) rather than waiting for the routine weekly Wikidata sync — closes the single most obvious staleness gap in the whole entity layer.

New table: elections

Field	Type	Description
id	uuid PK	
country_id	text FK	References countries.id
election_type	text	presidential parliamentary referendum other
scheduled_date	date	
status	text	upcoming completed disputed
result_summary	text	Nullable

SECTION 24 // ANALYST PANEL

LLM-powered analyst panel

A natural-language Q&A surface — 'what's the status of the Ukraine war?', 'what's going on in Palestine right now?' — grounded entirely in GlobeGrid's own data, not a generic chatbot bolted onto the side of the app. The distinction matters: this must answer from the fact chain, stories, and entity layer already built, with the same attribution discipline enforced everywhere else in this system (v1 Section 6.8) — never a freeform LLM answer with no link back to what it's based on.

24.1 UI concept: the orb, not a static sidebar

- Rather than a permanently visible chat sidebar competing with the globe for space, use a small persistent **orb** — a glowing point-of-light control, docked bottom-right, visually consistent with the globe/space motif already established (v2 Section 5.1's starfield/fresnel treatment). Idle state: a slow ambient pulse. It brightens briefly when the instability index (v1 Section 5.6) crosses a threshold, so it reads as 'alive' and aware of the world state, not just a static help button.
- Clicking/tapping the orb expands a slide-in panel from the right edge (~400px wide on desktop, full-width sheet on Tier 3 / mobile per v1 Section 11's tiering) — a conversational history above, a single-line input below. Closing collapses it back to the orb; nothing about the globe underneath is disrupted or resized.
- **Context-aware opening:** if the orb is opened while a specific country, conflict, or story is focused on-screen, the panel opens with that entity already silently scoped as context — so 'what's the status of this?' works without the user having to name what 'this' is. Opened with nothing focused, it behaves as a general query surface.
- Every answer that draws on specific stories/facts shows small inline citation chips beneath it — clicking one deep-links (v2 Section 5.3) straight to that story page. This is not optional polish; it's the same attribution requirement that governs every other surface in the app, applied to conversational answers.

24.2 Retrieval mechanism — two paths, mirroring the correlation engine's own design

Deliberately reuses the same two-path shape already established for correlation (v1 Section 5.4: same-window vs. historical-chain) rather than inventing a new retrieval philosophy.

- **Path 1 — structured entity match (fast, precise).** Parse the question for entities/regions using the same NER + canonical-entity resolution already built (v2 Sections 3.1-3.2). If the question resolves cleanly to a registered entity — a country, conflict, alliance, or non-state actor from the Section 13-23 entity layer — pull that entity's own live profile/synthesis directly as primary context. 'Status of the Ukraine war' resolves straight to the **conflicts** row (v3 Section 15) and its **summary** field — no search needed, just a direct lookup.
- **Path 2 — freeform retrieval (breadth, freshness).** Regardless of whether Path 1 hit, also run the question through the existing FTS5 search (v2 Section 6.4) and an embedding-similarity lookup (using the same embedder as correlation, v1 Section 5.4 step 2) against recent stories/facts, to surface anything current that a static entity profile wouldn't capture — this is what makes 'right now' answers actually current rather than only as fresh as the last profile synthesis.

- Both paths' results are merged into one context bundle passed to the answering call — structured-entity context grounds the answer, freeform results keep it current.

24.3 Answering discipline

- System prompt instructs the model to answer *only* from the retrieved context bundle — never silently falling back on general training knowledge when the fact chain has nothing relevant. If retrieval comes back empty or thin, the honest answer is 'GlobeGrid doesn't have current tracked data on that' rather than a plausible-sounding guess.
- Same confidence discipline as the causal linker (v1 Section 9) and the devil's-advocate pass (v3 Section 3): never state certainty the retrieved evidence doesn't support, and prefer 'as of the most recent tracked story...' framing over flat, timeless-sounding claims — geopolitical status can change quickly, and the answer should read like a snapshot, not a permanent fact.
- Every factual claim in the answer traces to a specific retrieved story/fact ID, stored alongside the message so the citation chips in 24.1 have something real to link to.

New table: `analyst_sessions`

Field	Type	Description
<code>id</code>	<code>uuid PK</code>	
<code>started_at</code>	<code>timestampz</code>	
<code>last_message_at</code>	<code>timestampz</code>	

New table: `analyst_messages`

Field	Type	Description
<code>id</code>	<code>uuid PK</code>	
<code>session_id</code>	<code>uuid FK</code>	References <code>analyst_sessions.id</code>
<code>role</code>	<code>text</code>	user assistant
<code>content</code>	<code>text</code>	
<code>cited_story_ids</code>	<code>text (JSON array)</code>	Nullable — populated on assistant messages only
<code>focused_entity_content</code>	<code>text</code>	Nullable — the entity in focus when this question was asked, per 24.1
<code>created_at</code>	<code>timestampz</code>	

Single-user system, so no per-user scoping is needed on these tables — same simplification already applied throughout (e.g. v2 Section 6.2's watchlists).

24.4 Autonomous navigation

Beyond answering, the panel can act on the UI itself — opening the relevant country or conflict tab, or flying the globe camera (v2 Section 5.1's fly-to tween) to a relevant marked location (v3 Section 17), when the question clearly points to one. Asking 'what's going on in Palestine right now?' doesn't just answer in text — it can jump the user straight to that conflict's tab, with the answer's citations now sitting directly beside the full live feed they support.

- Whichever entity Path 1 (24.2) resolved to — a country, conflict, alliance, non-state actor, or marked location — becomes a candidate navigation target, carried alongside the answer as structured data, not inferred separately from the text.
- A **settings toggle** (*Auto-navigate for me*) controls whether that candidate is acted on automatically or just offered: **on** — the UI navigates immediately when the answer streams in; **off** — the answer includes a clickable 'Open: [Conflict — Israel-Palestine] →' affordance instead, and nothing moves until the user taps it.
- This toggle changes *only* whether navigation is automatic. Citation chips (24.1) and the navigation affordance itself are never hidden by this setting — transparency about what the panel is grounded in is not optional, only the auto-jump behavior is.
- Default: **on**. This is meant to be the delight feature — asking a question and watching the app take you there — but a single persistent setting turns it off for anyone who finds unsolicited navigation disruptive, without losing any of the grounding/citation behavior.
- Guardrail: auto-navigation never fires for a question the panel answered with low confidence or empty retrieval (24.3) — jumping the user somewhere on a weak match is worse than not jumping at all.
- `analyst_messages.suggested_navigation` (text, JSON, nullable) — added to the schema in 24.3: `{type: country|conflict|alliance|non_state_actor|location, id: string}`.

One explicit scope boundary: if a question asks for a forecast ('will this escalate?'), the panel routes to the existing forecasting feature (v3 Section 7) rather than improvising a prediction inline — every forecast this system makes must go through the tracked, graded scorecard pathway (v2 Section 3.4), never a free-form answer that bypasses it.

SECTION 25 // CONFIG ADDITIONS

New config.yaml values

Appended to the config file as it stands after Sections 2-24 above — nothing prior is removed.

```

debate:
  personas: ["skeptic", "historian", "optimist"]
  disagreement_flag_threshold: 0.35

devils_advocate:
  enabled: true

embedding:
  model_name: "paraphrase-multilingual-MiniLM-L12-v2"

self_tuning:
  adjustment_step: 0.01
  min_threshold: 0.65
  max_threshold: 0.95
  aggregation_interval_hours: 24

anomaly:
  zscore_window_days: 30
  zscore_flag_threshold: 2.5
  cusum_enabled: true

forecasting:
  enabled: false          # explicit opt-in; off by default until scorecard has history
  min_resolved_predictions_for_high_confidence: 25
  default_horizon_hours: 72

satellite_overlay:
  tle_refresh_interval_hours: 24

provenance:
  hash_chain_enabled: true

geopolitical_entities:
  wikidata_refresh_interval_hours: 168    # weekly
  agenda_synthesis_interval_hours: 24     # ties to how often new tagged stories accumulate
  trade_stats_refresh_interval_hours: 720 # monthly, World Bank data changes slowly
  conflict_autotag_confidence_floor: 0.7  # below this, require manual confirm
  bilateral_relations_refresh_interval_hours: 48
  election_triggered_leadership_refresh: true

analyst_panel:
  enabled: true
  auto_navigate_default: true             # user-facing settings toggle; this is just the shipped default
  min_confidence_to_navigate: "medium"
  model: "claude-sonnet-5"                # can differ from CLAUDE_MODEL used for causal linking
  max_context_stories_per_query: 15
  freeform_retrieval_lookback_days: 30
  empty_retrieval_response: "no_current_data" # refuse to guess when retrieval is empty/thin
  session_history_retention: "indefinite"  # consistent with the fact chain's own never-delete policy

```

Note the explicit `forecasting.enabled: false` default — that feature ships turned off, consistent with Section 7's build-last, prove-it-first requirement.

SECTION 26 // BUILD SEQUENCE

Build sequence for Claude Code

Everything above is in scope. Sequencing follows the priority explicitly given when this brainstorm was proposed: lineage view and devil's advocate first (cheap, pure upside), hash-chained provenance third (near-free, and it's the dependency that makes forecasting credible later), forecasting deliberately last. The geopolitical entity layer (Phases S-W) is sequenced after the original legendary-tier phases since conflicts and non-state actors both depend on countries existing first.

v3 phases

Field	Type	Description
Phase J	Butterfly-effect lineage view	Section 8 — cheapest, most unique, pure upside; populate lineage_edges alongside existing historical_chain correlation writes
Phase K	Devil's-advocate pass	Section 3 — small, direct trust improvement, independent of everything else
Phase L	Hash-chained provenance	Section 11 — near-free, do this before Phase R so the scorecard has provenance from day one rather than retrofitted
Phase M	Story version history	Section 12 — small, independent, pairs naturally with Phase K since both concern narrative auditability
Phase N	Cross-lingual embedding swap	Section 4 — one config change + one re-embed pass; do once the v2 non-English sources have real volume flowing
Phase O	Multi-agent debate + anomaly detection	Sections 2 and 6 — independent of each other, both moderate builds
Phase P	Self-tuning thresholds	Section 5 — needs real user feedback volume to be meaningful; low urgency but cheap to wire up early and let it collect data
Phase Q	WebXR + generative music + timelapse + satellite overlay	Sections 9-10 — independent graphics features, batch together as a 'graphics v3' pass
Phase R	Forward-looking risk forecasting	Section 7 — deliberately last; requires Phase L (provenance) and a real resolved-prediction history from the v2 scorecard before forecasting.enabled is flipped true
Phase S	Country profiles + map/globe click + directory	Section 13 — the foundation for Phases T-V below; seed countries and boundaries first
Phase T	Alliances and blocs	Section 14 — depends on Phase S (countries must exist to be members of anything)
Phase U	Ongoing conflicts tabs	Section 15 — depends on Phase S; can build alongside Phase V since both feed conflict_parties
Phase V	Non-state actors	Section 16 — depends on Phase S (affiliated_state link) and pairs with Phase U (conflict party registration)
Phase W	Marked cities and locations + critical infrastructure	Section 17 — cheap and largely independent; capitals depend on Phase S, but the manually-curated chokepoint/infrastructure seed lists can be built anytime
Phase X	International organizations	Section 18 — independent of Phase S; register canonical entities and, where available, official press-release sources
Phase Y	Bilateral relations + sanctions + treaties	Sections 19-21 — all depend on Phase S (countries must exist); build together, they share the same source-linked synthesis pattern
Phase Z	Notable persons + elections tracker	Sections 22-23 — depends on Phase S and, for org-affiliated persons, Phase X; the election-triggered leadership refresh (23.1) should be wired in as soon as both exist

Field	Type	Description
Phase AA	Analyst panel	Section 24 — deliberately last in this document; Path 1 retrieval (24.2) depends on the entire entity layer (Phases S-Z) existing to resolve against, and autonomous navigation (24.4) has nowhere to navigate to until those pages exist

// END OF MANUAL — GLOBEGRID v3 LEGENDARY TIER — REV 3.3